Natural-Language Mathematical Statements to Formal Statements

Autoformalization as alignment, not translation

XMUM GE – Week 8

Ma Jiajun – May 26, 2026

Part I. Opening Case Study

One symbol, many formal objects

One Symbol, Many Formal Objects: Integral

On the board only this:

$$\int_a^b f(x)\,dx$$

> Question: to write this as a Lean theorem statement, what is missing?

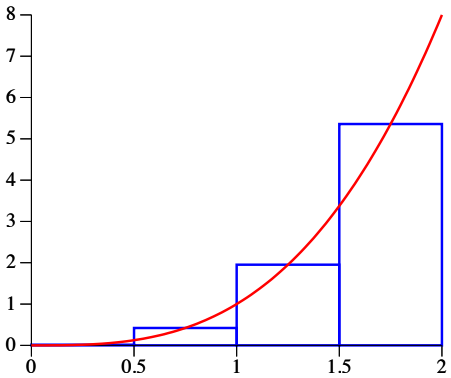
The missing piece is not Lean syntax. It is the mathematical object.

Possible readings

1. Riemann integral on a compact interval.
2. Lebesgue / Bochner integral wrt a measure.
3. Oriented interval integral $\int_a^b$.
4. Set integral over $[a, b]$, $(a, b]$, …
5. Variable upper limit $x \mapsto \int_a^x f(t)\,dt$.
6. Antiderivative relation $F' = f$.

Riemann vs Lebesgue: Two Ways To Sum The Area

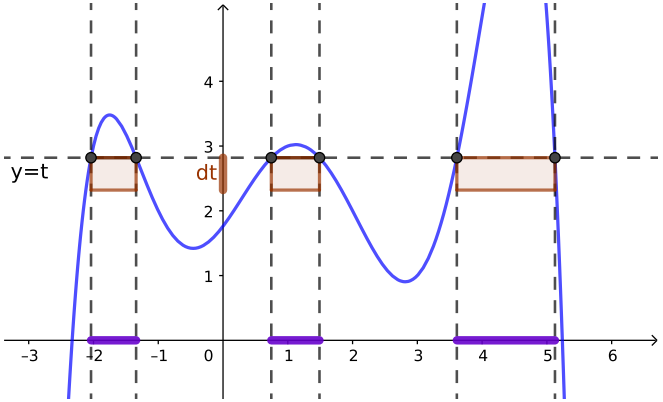
Riemann – slice the x-axis



$$\int f \approx \sum_i f(x_i)\,\Delta x_i$$

Cut the **domain** into vertical strips. Sum **height × width**.

Lebesgue – slice the y-axis



$$\int f \approx \sum_j y_j\,\mu\big(\{x : f(x) \geq y_j\}\big)$$

Cut the **range** into horizontal layers. Sum **value × measure of level set**.

Mathlib Says "As $R \to \infty$" With Filter and Tendsto

What the natural sentence says

$$\int_1^R \frac{\sin x}{x}\,dx \longrightarrow L \quad \text{as } R \to \infty$$

What Mathlib writes

```
Tendsto
  (fun R ⇒ ∫ x in 1..R, sin x / x)
  atTop
  (𝓝 L)
```

Three pieces — a function, a *source filter*, a *target filter*. No ε-δ on the surface.

A filter is "eventually"

A `Filter` on a type packages "what counts as *eventually*". The two we need:

> `atTop : Filter ℝ` — eventually, R is arbitrarily large.

> `𝓝 L : Filter ℝ` — eventually, the value lies in any neighborhood of L.

`Tendsto f F G` reads:

```
f pushes "eventually in F"
  to "eventually in G".
```

For our example: as R is eventually large, the partial integral is eventually near L. That is exactly

Running Example: Improper Riemann Exists, Lebesgue Does Not

$$\int_1^\infty \frac{\sin x}{x}\,dx \text{ exists.}$$

Reading A – improper Riemann integral: **true**

```
∃ L : ℝ,
  Tendsto
    (fun R : ℝ ⇒ ∫ x in (1:ℝ)..R, Real.sin x / x)
    atTop
    (𝓝 L)
```

Finite interval integrals have a limit as $R \to \infty$.

Reading B – Lebesgue integrable on $[1, \infty)$: **false**

```
¬ IntegrableOn
    (fun x : ℝ ⇒ Real.sin x / x)
    (Set.Ici 1)
```

$\int_1^\infty |\sin x/x|\,dx = \infty$. Lebesgue integrability needs absolute integrability; this is only conditional convergence.

Putnam 2025 B2: Geometry Becomes Integral Ratios

Let $f : [0, 1] \to [0, \infty)$ be strictly increasing and continuous. Let R be the region under $y = f(x)$ on $[0, 1]$. Let x_1 be the x-coordinate of the centroid of R. Let x_2 be the x-coordinate of the centroid of the solid generated by rotating R about the x-axis. Prove $x_1 < x_2$.

```
theorem putnam_2025_b2
    (f : ℝ → ℝ)
    (hf_cont : ContinuousOn f (Icc 0 1))
    (hf_mono : StrictMonoOn f (Icc 0 1))
    (hf_nonneg : ∀ x ∈ Icc (0 : ℝ) 1, 0 ≤ f x) :
    (∫ x in (0:ℝ)..1, x * f x) /
      (∫ x in (0:ℝ)..1, f x) <
    (∫ x in (0:ℝ)..1, x * (f x) ^ 2) /
      (∫ x in (0:ℝ)..1, (f x) ^ 2) := by
  sorry
```

The formal statement does not define R, centroid, or solid of revolution. It uses the analytic formulas directly.

Geometric narrative replaced by analytically equivalent integral ratios. The π cancels; denominator positivity is a proof obligation, not a hypothesis.

Gemini Draft: Correct Shape, Lean Syntax Fixes

Two layers on this slide

1. Apply Lean syntax fixes from earlier sessions: `import Mathlib`, `open Set Real MeasureTheory Interval`, `noncomputable` section, `Real.pi`, parameterize over `f`.
2. Once it typechecks, Gemini's **mathematical shape** is right.

```
noncomputable section

def area_R (f : ℝ → ℝ) : ℝ :=
  ∫ x in (0:ℝ)..1, f x

def x1 (f : ℝ → ℝ) : ℝ :=
  (∫ x in (0:ℝ)..1, x * f x) / area_R f

def vol_solid (f : ℝ → ℝ) : ℝ :=
  Real.pi * ∫ x in (0:ℝ)..1, (f x)^2

def x2 (f : ℝ → ℝ) : ℝ :=
  (Real.pi * ∫ x in (0:ℝ)..1, x * (f x)^2) /
    vol_solid f
```

Both versions express the same analytic comparison: AxiomMath inlines the integrals; Gemini names them. Presentation choice, not error.

"Indefinite Integral" Is Not A Defined Object

(A) Derivative side

```
HasDerivAt : (𝕜 → F) → F → 𝕜 → Prop
-- predicate: F' = f x at x
deriv : (𝕜 → F) → 𝕜 → F
-- total function; junk value 0 if no deriv
```

In `Mathlib.Analysis.Calculus.Deriv.Basic`. `HasDerivAt` is a `Prop`, `deriv` is a value.

(B) Definite integral side

```
intervalIntegral
  : (ℝ → E) → ℝ → ℝ → Measure ℝ → E
-- notation: ∫ x in a..b, f x ∂μ
```

In `Mathlib.MeasureTheory.Integral.IntervalIntegral.Ba` built on Bochner `MeasureTheory.integral`.

There is **no** `indefiniteIntegral` / `antiderivative` / `primitive` def in mathlib. Any "indefinite integral" must reduce to (A) or (B); FTC is the theorem connecting them, not a definition.

Three Informal Phrases, Two Primitives

Natural language	Real identity	Mathlib expression
"F is an antiderivative of f"	predicate on side (A)	`∀ x, HasDerivAt F (f x) x`
"the antiderivatives of f"	set carved out by that predicate	`{F | ∀ x, HasDerivAt F (f x) x}`
"$\int_a^x f(t)\,dt$"	a function on side (B)	`fun x ⇒ ∫ t in a..x, f t`

The first row uses **only** the derivative side. There is no integral in it.

The second row is a **set-builder**, not a `def`. mathlib has no `antiderivatives` definition.

The third row uses **only** the integral side. It is just a lambda over `intervalIntegral`.

Every alignment of "the indefinite integral of f" has to commit to one of three forms above – and therefore to one of the two primitives (A) or (B).

`deriv f x = 0` Is **Not** Evidence Of Differentiability

Core warning. `deriv f x = 0` does **not** imply that f is differentiable at x. The equation is **not** evidence of differentiability.

Why

`deriv` is a **total function**: Lean requires a value at every point.

If f is not differentiable at x, then `deriv f x` returns `0` by convention (junk value).

Two interpretations of `deriv f x = 0`

1. f *is* differentiable at x, derivative happens to be 0; **or**
2. f is *not* differentiable at x, `deriv` returned the default 0.

You cannot tell which from the equation alone.

Correct shape for "$f'(0) = 0$": use `HasDerivAt f 0 0` (packs existence + value together), or hold `DifferentiableAt ℝ f 0` as a separate hypothesis before reading off `deriv`. This is exactly why side (A) provides **both** `HasDerivAt` (assertion) and `deriv` (value).

Heaviside Step: The Junk-Value In Action

Setup

$$H(x) = \mathbf{1}_{[0,\infty)}(x)$$

```
def H : ℝ → ℝ :=
  Set.indicator (Set.Ici 0) (fun _ ⇒ 1)
```

H is **not continuous** at 0, so certainly not differentiable there.

Yet in Lean

```
example : deriv H 0 = 0 := by sorry
```

This holds – but via the **junk-value** path, not because the derivative equals 0.

`deriv H 0 = 0` ✓ (true)
`DifferentiableAt ℝ H 0` ✗ (false)
`HasDerivAt H 0 0` ✗ (false)

Reading `deriv f x = 0` as "the derivative at x is 0" is wrong whenever the function is not differentiable there. The compiler accepts the equation; the mathematical content has silently dropped out.

Fundamental Theorem Of Calculus As Statement Shapes

FTC-1

If $F(x) = \int_a^x f(t)\,dt$, then $F'(x) = f(x)$.

```
HasDerivAt
  (fun x ⇒ ∫ t in a..x, f t) (f x) x
```

FTC-2

If $F'(x) = f(x)$, then $\int_a^b f(x)\,dx = F(b) - F(a)$.

```
(∫ x in a..b, f' x) = F b - F a
```

Mathlib theorem names:

`intervalIntegral.integral_eq_sub_of_hasDerivAt`

`intervalIntegral.integral_deriv_eq_sub'`

The real content lives in the statement

- `HasDerivAt`, `HasDerivWithinAt`, or `deriv`?
- Where does the derivative hold?
- Integrability hypothesis?
- Oriented interval?
- Codomain structure?

"The fundamental theorem of calculus" $\Longrightarrow$ a family of theorem schemas.

Worked Example: $f' \equiv 0 \Longrightarrow f$ Constant (1/2)

Theorem (informal). If f is continuous on $[a, b]$ and $f' = 0$ on (a, b), then f is constant on $[a, b]$.

The "obvious" formal shape

```
theorem constant_of_deriv_zero
    {a b : ℝ} (hab : a < b) {f : ℝ → ℝ}
    (hcont : ContinuousOn f (Icc a b))
    (hdiff : DifferentiableOn ℝ f (Ioo a b))
    (hderiv : ∀ x ∈ Ioo a b, deriv f x = 0) :
    ∀ x ∈ Icc a b, ∀ y ∈ Icc a b, f x = f y :=
  sorry
```

Reads off the informal sentence directly: continuity on $[a, b]$, differentiability on (a, b), $f' = 0$ on (a, b), conclude constancy.

No Mathlib lemma has this exact shape

Closest closed-interval lemmas in `Mathlib.Analysis.Calculus.MeanValue`:

- `constant_of_derivWithin_zero`
- `constant_of_has_deriv_right_zero`
- `is_const_of_deriv_eq_zero` (no interval)

None accept "`DifferentiableOn` + `deriv = 0` on `Ioo a b`" as written. The alignment gap is in the **derivative notion**: at endpoints Mathlib uses `derivWithin` / `HasDerivWithinAt`, not `deriv` / `HasDerivAt`.

Worked Example: $f' \equiv 0 \Longrightarrow f$ Constant (2/2)

Real Mathlib API

```
-- closed interval ⟹ derivWithin, not deriv
example
    {a b : ℝ} {f : ℝ → ℝ}
    (hdiff : DifferentiableOn ℝ f (Icc a b))
    (hderiv : ∀ x ∈ Ico a b,
      derivWithin f (Icc a b) x = 0) :
    ∀ x ∈ Icc a b, f x = f a :=
  constant_of_derivWithin_zero
    hdiff hderiv
```

The lemma is `constant_of_derivWithin_zero`. Same lesson as the LeanArchitect / Multivariate Taylor case at the end of Part IV: at endpoints, use the `Within` variants.

Four alignment shifts vs. draft

- `DifferentiableOn` on `Icc` (continuity inherited)
- `derivWithin`, not `deriv`
- condition on `Ico a b`, not `Ioo a b`
- conclusion pinned to $f(a)$, not symmetric

The "deriv = 0 ⟹ constant" theorem has at least two real Mathlib shapes – both use the `Within` variants. **Neither** uses `deriv` or `HasDerivAt` directly.

Mathlib Does Not Start From Textbook Notation

Notation used by the Putnam B2 statement earlier:

```
∫ x in a..b, f x
```

comes from `MeasureTheory.Integral.IntervalIntegral`. Not Riemann sums as primary definition; an oriented interval integral built on the measure-theoretic integral.

```
∫ x in a..b, f x ∂μ
  = ∫ x in Ioc a b, f x ∂μ
  - ∫ x in Ioc b a, f x ∂μ
```

Orientation is part of the API:

```
∫ x in a..a, f x ∂μ = 0
∫ x in b..a, f x ∂μ
  = - ∫ x in a..b, f x ∂μ
∫ x in a..b, f x ∂μ
  + ∫ x in b..c, f x ∂μ
  = ∫ x in a..c, f x ∂μ
```

Formal libraries often choose definitions for theorem engineering, not for matching first-semester textbook wording.

Bochner Integral: Lebesgue For Vector-Valued Functions

Lebesgue you already saw

$$f : \mathbb{R} \to \mathbb{R}, \qquad \int_a^b f\,dx \in \mathbb{R}$$

The values are **real numbers**.

Concrete jump: a moving point in space

A particle's velocity is a vector that changes with time:

$$\mathbf{v} : [0, T] \to \mathbb{R}^3, \qquad \mathbf{v}(t) = (v_x(t), v_y(t), v_z(t))$$

Total displacement:

$$\int_0^T \mathbf{v}(t)\,dt \;\in\; \mathbb{R}^3$$

Bochner integral: same idea, any value type

$$f : \mathbb{R} \to E, \qquad \int f\,dx \;\in\; E$$

E can be $\mathbb{R}$, $\mathbb{R}^3$, $\mathbb{C}$, … – any space where "add vectors" and "measure size" make sense.

> When $E = \mathbb{R}$ the Bochner integral **equals** the Lebesgue integral. Nothing new.

Why does Mathlib write it like this?

So one definition covers every case:

```
E = ℝ    → real-valued     (sin x / x, FTC)
E = ℂ    → complex-valued  (Fourier)
E = ℝⁿ   → vector-valued   (velocity, force)
```

The Underlying Integral Is Bochner Integral

`MeasureTheory.integral` is the Bochner integral.

```
MeasureTheory.integral
  input:
    measure μ on domain α
    function f : α → E
    structure on E
      sufficient for integration
  output:
    element of E
```

Not only real-valued functions; takes values in suitable normed vector spaces.

Common notations:

```
∫ x, f x ∂μ
∫ x in s, f x ∂μ
∫ x in a..b, f x ∂μ
```

> Behind ordinary-looking notation sits a hierarchy:

```
measurable space / measure
  → integrable functions
  → Bochner integral
  → set integral
  → interval integral
```

Formalization step one: locate the concept in the library.

Mathlib Also Has Riemann-Style Integrals

Riemann-style integral lives in a separate module, `BoxIntegral`:

```
BoxIntegral.HasIntegral
BoxIntegral.Integrable
BoxIntegral.integral
BoxIntegral.IntegrationParams.Riemann
```

The same framework hosts Henstock–Kurzweil and McShane via different `IntegrationParams`.

Textbook phrase → Mathlib object:

```
"the Riemann integral"
  → BoxIntegral with
     IntegrationParams.Riemann

"the integral from a to b"
  → intervalIntegral built
     from MeasureTheory.integral
```

Bridge theorems connect box-style and measure-theoretic integrals under suitable hypotheses. The library keeps multiple related definitions; one is the more common proof interface.

What The Integral Case Teaches About Alignment

Template for the rest of the lecture:

```
informal phrase
  "the integral of f from a to b"

formal choices
  type of f
  codomain structure
  measure
  interval convention
  integrability condition
  endpoint behavior
  intended theorem shape
```

> *An informal mathematical statement is often a compressed reference to a whole formal design space.*

The integral example was concrete. Part II abstracts it into a general checklist.

Part II. General Mechanism

From informal sentence to formal statement

The Statement Is The Object

Week 8 builds:

> **a formal theorem statement**
> (not a proof)

Two checks on a formal statement:

1. It typechecks in the formal language.
2. It expresses the intended mathematical meaning of the informal statement.

> Dangerous autoformalization failures pass (1) but fail (2).

Example:

> Informal: *Every subgroup of a cyclic group is cyclic.*

A type-correct but wrong formalization could:

- state the ambient group is cyclic in the conclusion instead of the subgroup;
- use the wrong formal definition of "cyclic".

Alignment Checklist

1. **Objects.** Variables, functions, sets, structures, parameters.
2. **Types.** Each object's type.
3. **Structures.** Algebraic, order, topological, metric, measure, category.
4. **Quantifiers.** Universal, existential, order.
5. **Assumptions.** Explicit vs convention-hidden.
6. **Definitions.** Which library definition corresponds.
7. **Notation.** Overloaded symbols disambiguated.
8. **Statement strength.** Too weak, too strong, just right.
9. **Edge cases.** 0, empty set, equal endpoints, boundaries.
10. **Back-translation.** Translate the formal back; does it still read as the original theorem?

This is the human version of statement validation. Modern systems (Part IV) automate parts of it.

Typechecking Is Not Semantic Alignment

A Lean statement can be well-typed and still be the wrong theorem.

Common failure modes

- **Too weak.** Only a special case.
- **Too strong.** Extra hypotheses, or sharper conclusion.
- **Wrong object.** Nearby but different definition.
- **Wrong scope.** Quantifier order swapped.
- **Wrong ambient structure.** Specialized to $\mathbb{R}$ instead of generic.
- **Wrong convention.** Index from 0 vs 1; open vs closed interval; left derivative vs derivative.

> Formal statement checking is not enough. We also need statement alignment.

Part III. Worked Examples

Simple to research-level alignment problems

The First n Odd Numbers Sum To n^2

Informal:

> The sum of the first n odd numbers is n^2.

Hidden choices

- n is a natural number?
- "first n" means n terms, or up to n?
- Odd numbers as $2i + 1$ or $2i - 1$?
- Index $0..n - 1$ or $1..n$?
- Sum over `Finset.range`, list, recursion?
- Equality in `Nat`, `Int`, `Real`?

A possible Lean statement:

```
example (n : Nat) :
    (∑ i in Finset.range n, (2 * i + 1))
      = n ^ 2 := by
  ...
```

This statement has already made these choices:

- `n : Nat`;
- zero-based indexing;
- first n terms are $1, 3, \ldots, 2n - 1$;
- finite sum over `Finset.range`;
- equality in `Nat`.

Set Distributivity

Informal:

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

Hidden choices

- Universe type α?
- A, B, C are `Set α`?
- Extensional set equality?
- $\cap, \cup$ as set ops or lattice ops?

```
variable {α : Type*} (A B C : Set α)

example :
    A ∩ (B ∪ C)
      = (A ∩ B) ∪ (A ∩ C) := by
  ...
```

Even this simple example forces

- a universe `α`
- objects typed as `Set α`
- set lattice notation
- extensional equality

The Inverse Of A Product

Informal:

> The inverse of a product is the product of the inverses in reverse order.

Hidden choices

- Group, monoid-with-inverses, division ring, or category?
- Multiplication commutative or not?
- Conclusion $(ab)^{-1} = b^{-1}a^{-1}$?
- In a commutative setting, reverse order is hidden; otherwise substantive.

```
variable {G : Type*} [Group G]

example (a b : G) :
    (a * b)⁻¹ = b⁻¹ * a⁻¹ := by
  ...
```

> Natural language says "product" without naming the algebraic structure. Formal language cannot use multiplication until the structure is fixed.

Every Subgroup Of A Cyclic Group Is Cyclic

Informal:

> Every subgroup of a cyclic group is cyclic.

Hidden choices

- Ambient group G.
- Multiplicative vs additive notation.
- "Cyclic" as a predicate on groups, subgroups, or modules?
- Subgroup as bundled `Subgroup G`?
- Subgroup cyclicity uses the inherited group structure?

```
variable {G : Type*} [Group G]

-- sketch shape; final mathlib API
-- packages this differently:
∀ H : Subgroup G,
  IsCyclic G → IsCyclic H
```

> "Subgroup" in natural language refers both to a subset-like object and to a group in its own right. Formalization must choose the bundled representation that exposes the needed structure.

Continuous Functions On $[a, b]$ Attain A Maximum

Informal:

> A continuous function on $[a, b]$ attains a maximum.

Hidden choices

- $a \leq b$ assumed?
- $f : \mathbb{R} \to \mathbb{R}$, or $f : X \to \mathbb{R}$?
- `Continuous f` vs `ContinuousOn f (Set.Icc a b)`?
- "Attains a maximum" means $\exists x \in [a, b], \forall y \in [a, b], f(y) \leq f(x)$?
- Is this really compact-set extreme value?

```
∃ x ∈ Set.Icc a b,
  ∀ y ∈ Set.Icc a b, f y ≤ f x
```

A formal statement here specifies:

- domain set: `Set.Icc a b`;
- codomain order: $\leq$ on $\mathbb{R}$;
- continuity predicate: `ContinuousOn`;
- compactness source: closed bounded interval.

A common theorem name often hides compactness, order, and domain restriction.

$\mathrm{Gal}(E/\mathbb{Q}) \cong Q_8$: One Statement, Several Formal Shapes

Informal:

> Let $\alpha = \sqrt{(2+\sqrt{2})(3+\sqrt{3})}$ and $E = \mathbb{Q}(\alpha)$.
> Show $\mathbf{Gal}(E/\mathbb{Q}) \cong Q_8$.

Hidden choices

- α as `Real.sqrt …`;
- E as `IntermediateField.adjoin ℚ {α}`;
- $\mathbf{Gal}(E/\mathbb{Q})$ as `↥E ≃ₐ[ℚ] ↥E`;
- Q_8 as `QuaternionGroup 2`;
- $\cong$ as `≃*` (Type, not Prop).

```
-- Shape A: explicit MulEquiv (def, not theorem)
def gal_iso_Q8 :
  (↥E ≃ₐ[ℚ] ↥E) ≃* QuaternionGroup 2 := by
  sorry

-- Shape B: Galois-group predicate
theorem q8_is_galois_group
    [MulSemiringAction (QuaternionGroup 2) ↥E] :
    IsGaloisGroup
      (QuaternionGroup 2) ℚ ↥E := by
  sorry

-- Shape C: finite Galois group object
noncomputable def q8_as_finGaloisGroup_iso :
    Efin.finGaloisGroup
      ≅ FiniteGrp.of (QuaternionGroup 2) := by
  sorry
```

Three Shapes, Three API Paths

Shape A – MulEquiv

Direct group isomorphism.

```
(↥E ≃ₐ[ℚ] ↥E) ≃* QuaternionGroup 2
```

Closest to the informal $\cong$. Must use `def` (the type is `Type`, not `Prop`).

Shape B – IsGaloisGroup

Q_8 acts faithfully on E with fixed field $\mathbb{Q}$.

```
IsGaloisGroup (QuaternionGroup 2) ℚ ↥E
```

Mathlib bridges to `MulEquiv` via `IsGaloisGroup.mulEquivAlgEquiv`.

Shape C – FiniteGrp

Package E as `FiniteGaloisIntermediateField`.

```
Efin.finGaloisGroup ≅ FiniteGrp.of (QuaternionGroup 2)
```

"Finite Galois" lives inside the object, not as a separate hypothesis.

Three reasonable formal targets for one informal isomorphism. Choice of shape changes available API and downstream proof routes.

Does $\sum_{n=1}^{\infty} \frac{1}{n}$ Converge?

A one-line claim every undergraduate has seen. Try to formalize it. **What does "converge" formally mean?** The natural sentence is silent on six commitments at once.

Four hidden commitments

NL fragment	hidden commitment
$\sum_{n=1}^{\infty}$	partial-sum sequence vs limit object
$\frac{1}{n}$	$\mathbb{Q}$-, $\mathbb{R}$-, or $\mathbb{C}$-valued?
"converge"	classical limit, or extended-real $= +\infty$?
equality $= L$	$\exists L$ asserted, or specific L fixed?

Three Lean shapes – different theorems

```
-- A. partial-sum-tends-to-limit
∃ L : ℝ, Tendsto
  (fun N ⇒ ∑ k in Finset.range N, (1:ℝ)/(k+1))
  atTop (𝓝 L)

-- B. summability predicate
Summable (fun n : ℕ ⇒ (1:ℝ)/(n+1))
```

Answer (Euler, 1734)

$$\sum_{n=1}^{\infty} \frac{1}{n} = +\infty$$

Proof sketch – integral comparison:

$$\int_{1}^{N+1} \frac{dx}{x} \leq \sum_{n=1}^{N} \frac{1}{n}$$

LHS $= \ln(N+1) \to \infty$, so RHS $\to \infty$ too.

Part IV. Modern Autoformalization Systems

Mechanisms aimed at the alignment problem

What Modern Systems Are Trying To Fix

> *The formal statement **compiles**,*
> *and is **not** the intended theorem.*
> *This is the central failure mode every Part IV paper attacks.*

Five failure shapes seen in the wild

- **Free-variable swap** – `pi` ≠ `Real.pi`
- **Default-argument trap** – `(f := ...)` is a Lean default, not an asserted equation
- **Wrong object** – `Continuous` vs `ContinuousOn s`
- **Junk-value collapse** – `deriv f x = 0` even when f not differentiable (Slide 7)
- **Missing hypothesis** – silent drop of `[Fintype G]`

Each paper attacks one piece

Cog	Paper
Stronger acceptance	Rethinking – BEq
Library grounding	Rethinking, Aria, LoC
Self-critique	ReForm
No gold reference	Roundtrip
FL → NL infra	Herald / LeanSearch
Project scale	LeanArchitect

Rethinking: Typecheck Is Too Weak

> *Lean accepts the statement*
> *does **not** imply*
> *the statement is the intended theorem.*

The fix: BEq

Bidirectional Extended definitional Equivalence: both directions $S_{\text{gen}} \leftrightarrow S_{\text{ref}}$ must derive under a bounded set of rewrites.

Single direction can silently weaken the claim. Both directions cannot.

On 200 expert-labeled pairs

Metric	Prec.	Acc.
Typecheck	35	35
BLEU	63	69
LLM judge	71	83
BEq	**100**	**91**

BEq's recall is 73 % – the precision-100 zone. The gap above 73 % is where Aria / ReForm / Roundtrip do their work.

Rethinking: Retrieval Beats Generation

> *The bottleneck is not how well the LLM writes Lean.*
> *It is* ***which Mathlib objects sit in the LLM's context****.*

Build an informalized Mathlib

```
243,797 formal objects + 139,933 theorems
   ↓  bottom-up informalization
   ↓  along the dependency graph

each object carries:
   formal sig + NL description
   + list of dependencies
```

Then put the retrieved deps in the prompt

```
Old prompt:
  "Translate this theorem to Lean."

RAutoformalizer prompt:
  "Available Mathlib objects:
     Complex.differentiableOn, ...
   Now write the statement."
```

Con-NF BEq@8

```
baseline (no retrieval):     4.6 %
+ retrieval:                16.9 %     +12 pp
+ ORACLE deps (upper bound): ~35 %     +18 pp
```

The retrieval bump is half the climb to oracle. Plain semantic search misses the right Mathlib name two-thirds of the time (Recall@5 ≈ 35 % on ProofNet, 24 % on Con-NF).

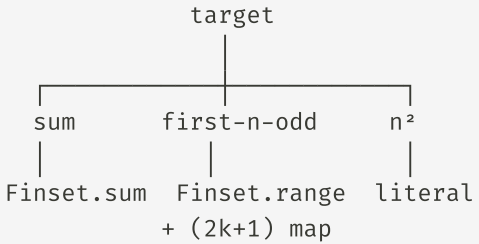
Aria: Concept Dependency Graph Before Lean Code

> *Never write Lean from NL directly.*
> *First decompose NL into a **concept graph**;*
> *ground the leaves in Mathlib;*
> *synthesize the missing inner nodes.*

The GoT pipeline (6 stages)

```
NL statement
  → concept dependency graph
  → ground leaves via LeanSearch
  → bottom-up synthesis of
    concepts missing from Mathlib
  → compiler-in-the-loop reflection
  → Lean theorem statement
```

Worked example: "first n odd numbers sum to n^2"



Every leaf grounds in Mathlib; inner nodes synthesized.

Headlines: ProofNet 68.5 %, FATE-H 71 %, homological conjectures 42.9 %. Ablations are decisive: **remove RAG → 0 %** on conjectures; **remove GoT → 6/14 → 1/14**. Both retrieval and graph structure are load-

AriaScorer: Term-Level Grounding

> *If a generated definition **compiles**,*
> *how do we know it is the intended **mathematical concept**?*

Mechanism

For every term in the candidate Lean statement, `jixia` static analysis retrieves from informalized Mathlib:

- name + kind + type signature
- def body
- informal name + description

A judge then compares assumptions and conclusions in this **grounded** context, not on the surface strings.

Three traps surface checkers miss

UFD / IsDomain. A surface checker flags "missing" `IsDomain`; AriaScorer sees that `UniqueFactorizationMonoid` already inherits it.

QuaternionGroup 1 vs 2. `QuaternionGroup n` in Mathlib has order $4n$. Surface match accepts either; AriaScorer reads the index – wrong for Q_8.

LoC-Decomp: A Template That Forces Commitments

> *Replace the one-shot theorem header*
> *with a **7-section template**.*
> *Each section forces a commitment NL hides.*

The seven sections

```
1. imports                ← which Mathlib API
2. aux types              ← what kinds of objects
3. mathematical systems   ← the unknowns
4. aux functions          ← helper notation
5. constraints            ← hypotheses
6. solution constraints   ← what "the answer" is
7. final statement        ← the theorem
```

Worked: "find all n with n^2+n+1 prime"

One-shot LLM:

```
theorem foo (n : ℕ) :
  Nat.Prime (n²+n+1) ↔ n = 1
```

– wrong: is $n = 0$ allowed? "find all" = set, predicate, or iff?

Template forces:

```
answer : Set ℕ
answer = {n | n ≥ 1 ∧ Nat.Prime (n²+n+1)}
theorem foo : answer = {1}
```

LoC-Decomp: ASCC + Numbers

ASCC = Automatic Semantic Consistency Check.
*Back-translate **per template section**, not whole file.*
Discrepancies localise to a Lean line.

Four-cell diagnostic

Compile	ASCC	Diagnosis
✓	✓	alignment evidence
✓	×	**wrong theorem proved**
×	✓	math right, API wrong
×	×	broken at both layers

The **(✓ compile, × ASCC)** cell is the failure mode NL → FL systems bleed through.

DeepSeek-V3 + LoC-Decomp pass rate

	MATH-500	miniF2F
raw LLM	~35 %	–
+ template	63 %	77 %
+ ASCC loop	**84 %**	**90 %**
Δ from loop	+21 pp	+13 pp

ASCC checker itself on 150 expert-labeled pairs: P = 0.90, R = 0.82, F1 = 0.86.

The loop is doing as much work as the template: structured decomposition alone leaves a 20-pp gap; only the per-segment feedback closes it. ReForm (next) moves the same loop *inside* the model via RL.

ReForm: Reflection Loop + PBSO Reward

> *The model generates, critiques itself, revises.*
> ***PBSO*** *rewards specific critiques, not just final answers,*
> *so the loop doesn't collapse into rubber-stamping.*

Architecture

```
Q → S₁ → C₁ → S₂ → C₂ → ...
        ↑       ↑
      reward  reward
```

Each C_t should **cite Q's structure**, **name a missing commitment**, **surface an ambiguity**.

Reward split

$$\mathcal{L} = -\mathbb{E}\left[R_{\text{task}} + \lambda \sum_t R_{\text{aux}}(t)\right]$$

Shallow vs specific critique (FTC example)

Shallow: "Syntax is valid; the statement matches the problem." → $R_{\text{aux}} \approx 0.1$

Specific: "Q is conditional 'if $F' = f$, then …'. S_1 never introduces F as antiderivative. Missing: `HasDerivAt F (f x) x`; integrability of f. Also: `HasDerivAt` vs `HasDerivWithinAt` unspecified."

ReForm: Numbers + Audit Of "Gold" Benchmarks

Headline (semantic-consistency %)

Model	miniF2F	ProofNet	Putnam	AIME
Baseline best	73	48	36	22
ReForm-32B	**91**	**70**	**62**	**67**
Δ	+18	+23	+27	+45

Pattern: **harder benchmark, larger lift**. Average +22.6 pp.

The audit finding

Running ReForm's ConsistencyCheck (859 items) on **human-written** gold benchmarks:

> miniF2F: **16.4 %** have semantic errors.
> ProofNet: **38.5 %** have semantic errors.

Three caught examples:

- $\sqrt{11}$ rewritten as 11 in Lean
- $\deg p \leq 80 \to \deg p < 80$
- "find X" baked into "prove $X = c$"

If gold references themselves carry 16–39 % errors, "compare to reference" is not a clean ground truth. **Roundtrip** (next) drops the gold-reference assumption entirely.

Roundtrip: Verification Without A Gold Target

> *Real autoformalization has **no gold** Lean reference (novel math, Mathlib PRs, legal code, ...).*
> *Use NL → FL → NL → FL stability as the signal.*

The roundtrip

```
x        informal statement
y_orig = formalize(x)          T1
x'     = informalize(y_orig)   T2
y_rt   = formalize(x')         T3

check  y_orig  ≡  y_rt
```

If $y_{\text{orig}} \not\equiv y_{\text{rt}}$, drift entered one of T1, T2, or T3.

Scoped repair – only the broken stage regenerates

Diagnosis	Repair
T1 broke	re-formalize, T2 & T3 re-run
T2 broke	re-informalize, only T3 re-runs

Three debug questions for any disagreement

1. Was the first Lean statement wrong? (T1)
2. Was the back-translation misleading? (T2)
3. Was the second formalisation unstable? (T3)

Parallel to LoC-Decomp

Both **localise the failure**; that is half their value.

Method	Localises by
LoC-Decomp	template *segment*
Roundtrip	pipeline *stage*

Roundtrip: Numbers + Wrong Fixed Points

Statute-code accuracy (Z3 equivalence)

Texas Transportation Code / Parks & Wildlife:

```
No repair:                  45 / 66 %
Claude + scoped repair:     85 / 86 %
Best (GPT + Claude diag):   91 / 95 %
```

The diagnoser is the lever

GPT default prompt blames T1 in **83 % / 97 %** of cases — nearly always. A non-sequential prompt drops to **9 % / 16 %**, and end-to-end accuracy rises. **Same models, only the diagnoser prompt changed.**

The wrong fixed point

If T2 drops the *same* hypothesis as T1, the roundtrip stabilises on it – a wrong $y_{\text{orig}} \equiv y_{\text{rt}}$ that looks correct but isn't.

Mitigation: multiple T2 paraphrase paths x'_1, x'_2, x'_3. Confidence rises only if all paths agree.

Why statute, not Lean

Z3 decides equivalence of FOL rules; Lean equivalence often needs Mathlib lemmas (sometimes a new theorem). Same idea, harder checker.

Why The Reverse Direction Matters: Formal → Informal

Part IV so far has been NL → FL. The reverse direction — turning Mathlib's 200 K declarations into NL — is the ***upstream infrastructure*** *behind everything we've seen.*

Four uses for FL → NL

1. Training data. Mathlib4: ~200 K formal decls, ~0 paired NL. Herald produces 44 k theorem pairs, 45 k proof pairs — all by back-translation.

2. Retrieval. LeanSearch, Aria leaf-grounding, ReProver premises — all index the *informalized* Mathlib, not the raw source.

3. Verification. ReForm critique, Roundtrip, Herald validator all back-translate $S \to Q'$ and compare $Q \approx Q'$. No FL → NL → no check.

4. Documentation. 1.5 M lines of Mathlib become navigable. Same FL → NL component powers docs, error messages, tutorials.

FL → NL is **not symmetric** with NL → FL. It is the prerequisite that makes the rest of Part IV possible.

Herald: Mathlib4 → Natural-Language Dataset

> *Bare LLMs reading Mathlib source **reproduce identifiers** (`Set.Ico`) instead of unfolding concepts.*
> *Herald wraps the LLM with structured Mathlib metadata.*

Pipeline (sketch)

```
Mathlib4
  ↓  jixia static analysis
     (names, types, deps, instances,
      proof states)
  ↓  dependency-aware informalization
  ↓  Herald dataset + Translator
```

Scale

Asset	Count
Valid statements	580 k
NL ↔ FL theorem pairs	44 k
NL ↔ FL proof pairs	45 k

`Set.Ico` walkthrough

```
theorem foo (a b c : ℝ) :
  c ∈ Set.Ico a b ↔ a ≤ c ∧ c < b
```

Naive LLM: "c belongs to **Ico** from a to b iff a ≤ c and c < b." – identifier leaks.

Herald: "c lies in the **left-closed right-open interval [a, b)** iff a ≤ c < b." – chained, named.

Same LLM, same statement. The difference is **structured metadata fed to the prompt**, not model capacity.

LeanSearch: Informalized Mathlib As A Service

> *Herald's dataset becomes a **query service**.*
> *NL question in → relevant Mathlib declaration out.*

How it's used

```
NL: "continuous function on
     a compact set attains
     a maximum"
  ↓ embedding model
     trained on Herald pairs
  ↓ nearest-neighbour over
     informalized Mathlib
  ↓ Mathlib decls:
       IsCompact.exists_isMaxOn
       ContinuousOn.isCompact_image
```

Downstream

Consumer	Use
LeanSearch	NL → Mathlib API search
Loogle	type-pattern search using NL labels
Aria / ReProver	retrieval inside NL → FL

Deployed at `leansearch.net`.

FL → NL is **not symmetric** with NL → FL. It is the **upstream infrastructure** behind every "RAG over Mathlib" pipeline in Part IV.

Herald Translator: Numbers + Two Case Studies

Coverage by domain

```
miniF2F-test                96.7 %
internal graduate text      23.5 %
College CoT                 16.0 %
```

Contest math: nearly solved. Graduate math: much harder.

Validation pipeline

```
formal candidate
  → Lean compile check
  → back-translate to NL
  → NLI check vs original
```

Case 1: quantifier order ($\forall x \exists \delta$ vs $\exists \delta \forall x$)

"f continuous on X" → two non-equivalent Lean shapes:

```
∀ x, ∀ ε, ∃ δ, ∀ y, …      -- continuity
∀ ε, ∃ δ, ∀ x, ∀ y, …      -- uniform continuity
```

Heine–Cantor: B follows from A on compact X; strictly stronger otherwise. Moving x past $\exists \delta$ silently rewrites the source.

Case 2: Four Lemma (mono half)

Lean header full of `ComposableArrows.app' φ i …` and index proofs. Faithful NL must restore *"morphism between exact chains; if φ_0 epi and φ_1, φ_3 mono, then φ_2 mono"* – without unfolding the index machinery.

What Makes Formal → Informal Hard: Three Tax

> *FL → NL doesn't collapse into "ask an LLM to read Lean code".*
> *Three independent taxes any back-translator must pay.*

1. Naming tax

Mathlib identifiers = encyclopaedic labels.

```
Set.Ico → left-closed
  right-open interval
StrictMono → strictly inc.
IsCompact → compact
Mono / Epi → mono / epi
```

Need a learned naming map.

2. Type-class tax

Which implicit args surface in NL?

- `[Abelian C]` → yes
- `[DecidableEq G]` → no
- `[CommRing R]` → yes
- universe vars → no

No fixed rule; per Mathlib design.

3. Surface tax

Lean syntax is dense:

- chain: $a \le c \wedge c < b \to a \le c < b$
- bracket notation
- `CategoryTheory.Abelian.mono_of_…` → "Four Lemma"
- $\mathbb{R}$ vs `Real`

Beyond Theorem Headers: Three Levels Of FL → NL

*Herald and LeanSearch target **theorem statements**.*
Proofs and definitions are harder; the alignment problem reopens at each level.

Level 1: statement

```
theorem ContinuousOn.isCompact_image
  ... (hf : ContinuousOn f s)
      (hs : IsCompact s) :
  IsCompact (f '' s)
```

→ "Continuous function on compact set has compact image."
Status: Herald 96.7 % on miniF2F, 16 % on graduate.

Level 2: proof

```
intro ε hε
obtain ⟨δ, hδ, h⟩ := ...
refine ⟨δ, hδ, ...⟩
calc |f y - f x| < ε / 2 ...
  _ < ε := by linarith
```

→ NL prose with **proof-state context** at every step.
Status: Lean-STaR partial; mostly open.

Level 3: definition

```
class CompleteLattice (α) extends ...
  sSup : Set α → α
  le_sSup : ...
  sSup_le : ...
```

→ "A lattice with arbitrary suprema."
Status: Mathlib docstrings, hand-written; no scalable generation.

Most "autoformalization needs autoinformalization" claims in Part IV assume **Level 1**. Levels 2 and 3 reopen

LeanArchitect: `@[blueprint]` – One Source Of Truth

> *Big Lean projects keep a LaTeX blueprint plus Lean code.*
> *Two artifacts drift; **proof agents trip over the drift**.*
> *LeanArchitect makes Lean the single source – LaTeX is derived.*

The Lean attribute

```
@[blueprint "thm:add-comm"
  (statement := /-- Addition is
                   commutative. -/)
  (uses := [def:nat, lem:zero-add])]
theorem MyNat.add_comm ...
```

Holds the LaTeX label, NL statement, declared deps. The extractor reads constants actually used and emits `\leanok`, `\uses{ ... }`, `\notready` automatically.

Drift modes auto-sync kills

- stale prose vs Lean header → edited together
- missing/wrong `\uses{ ... }` → extractor cross-checks
- wrong `\leanok` → computed, not asserted
- divergent edits → only one file to edit

Caveat. Auto-sync $\ne$ semantic equivalence. `@[blueprint]` closes *metadata* drift. Whether the NL statement and the Lean statement still mean the same thing is the alignment problem Aria / LoC / ReForm

LeanArchitect Case: Multivariate Taylor And `derivWithin`

> *Project-scale alignment bugs look like **proof failures**.*
> *The root cause is almost always a **statement upstream**.*

The workflow

```
GPT-5 drafts blueprint + sorrys
   ↓
LeanArchitect builds graph
   ↓
Aristotle proves leaves
   ↓
Human inspects failures
```

What happened

~40 nodes; **37 discharged**; **3 stuck**. All three stuck on lemmas about derivatives *on the segment* $[a, b]$.

The bug – not a proof problem

The Lean used `deriv` for a function on a segment $[a, b]$.

Correct formal object: `derivWithin (Icc a b)`.

Outside the segment, `deriv` returns junk; the statement was never the segment-restricted theorem.

Without the graph: "Aristotle is weak here". With the graph: trace upstream → fix the statement, not the prover.

Part IV Synthesis

> *Seven papers. One diagnosis:* ***typecheck is too weak****.*
> *Each paper attacks one cog of a larger alignment machine.*

Better acceptance

What the statement *means*, not just whether Lean compiles.

- BEq (precision 1.00)
- AriaScorer (term grounding)
- ConsistencyCheck audit

Library grounding

Autoformalization is *retrieval + generation*.

- RAutoformalizer (+12 pp)
- Aria GoT pipeline
- Herald + LeanSearch (infra)

Feedback + scale

Alignment is a process, not a one-shot.

- LoC ASCC loop (+20 pp)
- ReForm + PBSO (+22.6 pp avg)
- Roundtrip (no gold)
- `@[blueprint]` at project scale

Part V. Alignment Template

Fields a formal statement must pin down

Fields A Formal Statement Must Pin Down

```
Informal statement:

Objects:

Types:

Structures / typeclasses:

Definitions from library:

Quantifiers:
```

```
Assumptions:

Conclusion:

Edge cases:

Back-translation:
```

> The deliverable is not a proof. It is a formal statement whose every field above has been committed to deliberately.

Sample Informal Claims For Alignment

A. The inverse of a product is the product of inverses in reverse order.

B. A continuous function on $[a, b]$ attains a maximum.

C. Every subgroup of a cyclic group is cyclic.

D. The first n odd numbers sum to n^2.

E. The indefinite integral of f is F.

Three possible formal targets: derivative-side predicate, derivative-side set, or a specific definite-integral-with-variable-upper-limit. Each gives a different theorem.

Alignment Audit: Six Questions For Any Formal Statement

Six questions to check a formal statement S against an informal claim Q:

1. If S is true, does Q follow?
2. If Q is true, does S follow?
3. Was an assumption silently added that wasn't there in Q?
4. Was a needed assumption dropped?
5. Was the object swapped (e.g. ambient group instead of subgroup, `deriv` instead of `derivWithin`)?
6. Does a back-translation of S still sound like Q?

> "Yes" on (1) and (2), "no" on (3)–(5): the minimum bar for semantic alignment. Typecheck does not give this for free.

Part VI. Mathematical Sidebars

Harmonic series, primes, and density

Standalone math; not assessed.

Leonhard Euler (1707–1783)

Handmann pastel portrait, 1753 (Kunstmuseum Basel, public domain)

What he is to this section

- Proved $\sum_{n=1}^{\infty} \frac{1}{n} = \infty$ – the harmonic series diverges
- Discovered the **Euler product** $\zeta(s) = \prod_p (1 - p^{-s})^{-1}$ for $\Re(s) > 1$
- Gave the first analytic proof that there are infinitely many primes: $\sum_p \frac{1}{p}$ already diverges
- Standardised much of modern math notation: $\sum$, e, i, π, $f(x)$

Why he shows up at Week 8

Euler's identity $\sum_p \ln \frac{1}{1-1/p} = \sum_n \frac{1}{n}$ is one line of math that hides a type choice (real vs complex), a convergence condition ($\Re(s) > 1$ or analytic

Harmonic Series: $\sum \frac{1}{n}$ Diverges

Integral comparison

$$\sum_{n=1}^{N} \frac{1}{n} \geq \int_{1}^{N+1} \frac{dx}{x} = \ln(N+1)$$

As $N \to \infty$, the right-hand side $\to \infty$, so the partial sums $\to \infty$ as well.

Cauchy's blocking proof

Group terms by powers of 2:

$$\sum = 1 + \tfrac{1}{2} + \underbrace{\left(\tfrac{1}{3} + \tfrac{1}{4}\right)}_{\geq 1/2} + \underbrace{\left(\tfrac{1}{5} + \cdots + \tfrac{1}{8}\right)}_{\geq 1/2} + \cdots$$

Infinitely many blocks each $\geq \frac{1}{2} \Rightarrow$ total is ∞.

Numerical taste of the divergence

N	$\sum_{n=1}^{N} 1/n$
10	2.929
100	5.187
1 000	7.485
10 000	9.788
10^9	≈ 21.30

Growth is logarithmic. Euler's constant $\gamma = \lim(H_N - \ln N) \approx 0.5772$ – never proved irrational, still open.

> The series diverges, but **slowly**. This is the heart of *conditional* rearrangements (Riemann's rearrangement theorem) – slow divergence is what

Alternating Harmonic: $\sum \dfrac{(-1)^{n-1}}{n} = \ln 2$

Convergence by Leibniz's criterion

If a_n is positive, decreasing, and $a_n \to 0$, then $\sum (-1)^{n-1} a_n$ converges.

Here $a_n = 1/n$ satisfies all three: positive, decreasing, $\to 0$. So $\sum_{n=1}^{\infty} \frac{(-1)^{n-1}}{n}$ converges.

Identifying the limit

From the Taylor series

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \cdots, \quad |x| < 1$$

evaluate carefully at $x = 1$ (Abel summation, since the series is at the boundary of convergence):

Conditional vs absolute

$$\sum_{n=1}^{\infty} \left| \frac{(-1)^{n-1}}{n} \right| = \sum \frac{1}{n} = \infty$$

So the alternating sum converges, but **not absolutely**. The convergence depends on the sign pattern.

Riemann's rearrangement theorem. Because the convergence is conditional, you can *rearrange* $\sum (-1)^{n-1}/n$ to converge to **any real number you want**, or to $\pm\infty$, or to diverge. The "value" $\ln 2$ depends on the order.

Euler's Proof: Infinitely Many Primes

Recorded by Euler around 1737. The first analytic proof – uses the harmonic series instead of Euclid's combinatorial argument. Stronger conclusion: $\sum_p \tfrac{1}{p}$ itself diverges, so primes are "logarithmically dense".

The Euler product

For $s > 1$ define

$$\zeta(s) \;=\; \sum_{n=1}^{\infty} \tfrac{1}{n^s}$$

By unique factorization,

$$\zeta(s) \;=\; \prod_{p} \tfrac{1}{1-p^{-s}}$$

– the product runs over all primes p. This is the **Euler product** identity.

From the product to the prime series

Take logs of the product:

$$\sum_{p} \ln \tfrac{1}{1-1/p} = \infty$$

Use $\ln \tfrac{1}{1-x} = x + \tfrac{x^2}{2} + \tfrac{x^3}{3} + \cdots$, so for $x = 1/p$:

$$\sum_{p} \tfrac{1}{p} \;+\; \underbrace{\sum_{p} \tfrac{1}{2p^2} + \tfrac{1}{3p^3} + \cdots}_{\text{converges (bounded by } \sum 1/n^2)} \;=\; \infty$$

Bernhard Riemann (1826–1866)

Portrait c. 1863 (public domain)

What he is to this section

- Extended Euler's $\zeta(s)$ to a **meromorphic function on all of** $\mathbb{C}$ with one simple pole at $s = 1$
- 1859 memoir "*Über die Anzahl der Primzahlen unter einer gegebenen Grösse*" connected ζ's zeros to the distribution of primes
- Stated the **Riemann Hypothesis**: all non-trivial zeros lie on $\Re(s) = 1/2$ – still open after 167 years
- Foundational complex analysis: Riemann surfaces, the **identity theorem** (uniqueness of analytic continuation)

Meromorphic Functions And The Identity Theorem

What "meromorphic" means

A function $f : U \subseteq \mathbb{C} \to \mathbb{C}$ is **meromorphic** on an open connected domain U if:

1. f is holomorphic on U except at isolated singularities;
2. every singularity is a **pole** (not essential), meaning near z_0:

$$f(z) = \sum_{k=-m}^{\infty} a_k (z - z_0)^k$$

with finitely many negative-index terms ($m \geq 0$, the order of the pole).

Equivalently: $f = g/h$ where g, h are holomorphic and $h \not\equiv 0$.

The identity theorem (uniqueness)

> If f and g are meromorphic on a connected open U and they **agree on a subset $S \subset U$ with a limit point in U**, then $f = g$ on all of U.

In particular: **a meromorphic extension is unique**. Once we have ζ defined for $\Re(s) > 1$ by the series, there is at most one meromorphic function on $\mathbb{C}$ that agrees with it there. Riemann's continuation gives *the* extension.

Why this matters

Peter Gustav Lejeune Dirichlet (1805–1859)

Public-domain portrait

What he is to this section

- **Dirichlet's theorem (1837):** for any coprime a, q, the progression $a, a+q, a+2q, \ldots$ contains infinitely many primes
- Introduced **Dirichlet characters** $\chi : (\mathbb{Z}/q\mathbb{Z})^\times \to \mathbb{C}^\times$ and **Dirichlet L-functions** $L(s,\chi) = \sum \chi(n)/n^s$ – the engine of the proof
- **Dirichlet density**, a refined alternative to natural density that survives where natural density doesn't exist
- The first to make Euler's analytic methods rigorous; succeeded Gauss at Göttingen in

Dirichlet's Theorem And Density

Dirichlet's theorem on primes in APs (1837)

Let $a, q \in \mathbb{Z}$ with $q \geq 1$ and $\gcd(a, q) = 1$. Then the arithmetic progression

$$a,\ a+q,\ a+2q,\ a+3q,\ \ldots$$

contains **infinitely many primes**.

Hidden commitments in this statement

NL fragment	hidden commitment
"a, q coprime"	$\gcd(a, q) = 1$ in which ring?
"arithmetic progression"	sequence, set, or residue class?
"infinitely many primes"	infinite set vs co-finite
"primes"	`Nat.Prime` or UFM-prime?

Equidistribution refinement

Two density definitions – non-equivalent

For a set S of primes:

Natural density

$$d_{\text{nat}}(S) = \lim_{x \to \infty} \frac{\#\{p \in S : p \leq x\}}{\pi(x)}$$

(when the limit exists; $\pi(x)$ counts all primes $\leq x$).

Dirichlet density

$$d_{\text{Dir}}(S) = \lim_{s \to 1^+} \frac{\sum_{p \in S} p^{-s}}{\sum_{p} p^{-s}}$$

(when the limit exists).

Why These Three Topics Sit Together